INT4418 · DỮ LIỆU LỚN · BÀI TẬP LỚN

Autoformalization Pipeline

Sinh và kiểm chứng hàng loạt phát biểu Lean 4 từ đề toán

```
theorem cand_02e98146b414 (kyle tasha : ℕ)
  (h₁ : 8 + kyle + tasha = 36) (h₂ : kyle = tasha) :
  kyle = 14 := by sorry
```


NHÓM 5

Nguyễn Mai Hạnh B25CHHT093

Lê Quang Đức B25CHHT086

Nguyễn Việt Hưng B25CHHT099

Kim Anh Dũng B25CHHT087

Bùi Giang Linh B25CHHT103

Vũ Quốc Hùng B25CHHT098

Giảng viên: TS. Nguyễn Kiều Linh · Học viện Công nghệ Bưu chính Viễn thông

BÀI TOÁN

Dịch đề toán sang Lean 4, rồi để Lean kiểm chứng

ĐỀ GSM8K · NGÔN NGỮ TỰ NHIÊN

Carla, Kyle, and Tasha caught 36 fish. If Kyle and Tasha caught the same number of fish and Carla caught 8, how many fish did Kyle catch?

Answer: 14



LLM
zero-shot

PHÁT BIỂU LEAN 4

```
theorem cand_02e98146b414
  (kyle tasha : ℕ)
  (h₁ : 8 + kyle + tasha = 36)
  (h₂ : kyle = tasha) :
  kyle = 14 := by sorry
```

COMPILE_OK · 12,7 ms

Lean tự suy ra đáp án 14

Lean bảo đảm phát biểu đúng kiểu, không bảo đảm đúng đề.

HAI CÂU HỎI NGHIÊN CỨU CỦA ĐỀ BÀI

1 Batching và số candidate ảnh hưởng thế nào tới throughput, chi phí và compile rate?

Gom bài ở tầng LLM, batch size ở tầng Lean, 1 so với 4 candidate · slide 7–9

2 Lean chấp nhận nhưng phát biểu sai ý đề: phát hiện thế nào?

Ba lớp kiểm tra, audit 100 mẫu, mutation test · slide 12–13

Câu hỏi của slide môn học, "vai trò Big Data được chứng minh bằng thí nghiệm nào?", trả lời bằng ma trận 100 → 10.000 task, đường cong worker và thí nghiệm chịu lỗi · slide 10, 11, 14

DỮ LIỆU

22.150 đề thật từ bốn nguồn công khai

Nguồn	Số đề	Loại bài	Có sẵn
ProofNet	371	Định lý đại học	Lean gold
miniF2F	488	Thi Olympic	Lean gold
MATH	12.499	7 môn, mức 1–5	Đáp án
GSM8K	8.792	Toán lời văn	Đáp án

Tải từ GitHub, chuẩn hóa về một schema, loại 1 bản ghi trùng. NuminaMath-CoT không tải được trong môi trường thí nghiệm, nên dùng MATH và GSM8K, hai nguồn con của nó. Gold viết cho Mathlib 2024: 84,9 % còn compile trên v4.35, 92,9 % sau một luật viết lại cú pháp.

ĐƠN VỊ DỮ LIỆU

`DataUnit = ⟨problem_id, source, natural_statement, domain, metadata⟩`

KHOÁ PHÂN VÙNG

`SHA256(problem_id) mod 16`

Mọi máy tính ra cùng một partition, khác hash() của Python

BA TẦNG THÍ NGHIỆM

T100 ⊂ T1K ⊂ T10K, cùng tỷ lệ nguồn, seed 42; T20 cho baseline

ỨNG VIÊN LEAN

1.420 phát biểu do Claude Haiku 5.5 sinh

1.000 zero-shot + 100 × 4 few-shot + 20 baseline một request; tầng 10K phát lại 1.000 ứng viên thật 10 lần

Chi tiết: docs/DATA_CARD.md · data/MANIFEST.json (nguồn, giấy phép, SHA-256) · data_sample/

KIẾN TRÚC

Bốn tầng nối với nhau bằng hàng đợi phân vùng

Queue

Hàng đợi đề

16 partition; worker lấy việc theo lease

Batch Inference

Sinh ứng viên

LLM sinh k phát biểu mỗi bài, cache theo TaskHash

Lean Verify Pool

Kiểm chứng Lean

Mỗi worker giữ một REPL đã nạp Mathlib; heartbeat + timeout

Store / Aggregator

Tổng hợp

Pass@k, P95, RAM; corpus chọn lọc + kiểm tra ngữ nghĩa

HỢP ĐỒNG GIỮA CÁC WORKER

TaskHash

`SHA256(problem_id || prompt_version || candidate_index)`

Khoá của mọi task và kết quả

Lease, thử lại ≤ 3

Worker chết thì lease 60 s hết hạn, task quay lại hàng đợi; quá 3 lần vào dead-letter

UPSERT

Ghi theo TaskHash: chạy lại bao nhiêu lần cũng chỉ một dòng kết quả

Hai broker

SQLite WAL trên một máy; Redis Streams cho nhiều máy, cùng interface

Đủ 7 thành phần gợi ý: queue · prompt/version · batch inference · candidate store · Lean pool · result store · audit

THIẾT LẬP THÍ NGHIỆM

Một baseline, tám yếu tố, hai metric chính

BASELINE BẮT BUỘC

Zero-shot · 1 bài mỗi request · 1 candidate · 1 Lean worker · không repair

20 request riêng trên T20; cấu hình kiểm chứng A trên 100, 1.000, 10.000 bài. Cùng dữ liệu, split, máy và timeout với phương pháp chính.

HAI METRIC CHÍNH

1 · Độ chính xác ngữ nghĩa trên tập audit

2 · Throughput ổn định, ứng viên/s

Phụ: parse, compile, Pass@k, P50/P95, token, lỗi theo loại

Yếu tố	Mức	Lần chạy
Quy mô × cấu hình	100 · 1K · 10K × A, B, C	27
Batch size kiểm chứng	1 · 8 · 32, lô 1 có index	36
Số Lean worker	1 · 2 · 4 · 8	6
Bài mỗi request LLM	1 · 9 · 20–26 · 100–142	64 request
Số candidate	1 · 2 · 3 · 4	400 ứng viên
Prompt	zero-shot · few-shot	2
Retry khi giết worker	giao tối đa 1 · 3 lần	6
Release khi khởi động lại REPL	bật · tắt	6

3 lần mỗi ô, báo median và min–max; seed 42; heartbeat 400000; wall timeout 10 s hoặc 30 s

run_record.json cho mỗi lần chạy: commit, dữ liệu, máy, phần mềm, seed, tham số, metric, log · configs/experiments.json

BASELINE VÀ CHẤT LƯỢNG SINH MÃ

Compile gần như bão hòa: 98,9 % trên 1.000 bài

98,9 %

989/1.000 phát biểu zero-shot compile được; parse rate 99,7 %

Nguồn	Compile	Tỷ lệ
ProofNet	35 / 37	94,6 %
miniF2F	49 / 49	100 %
MATH	805 / 814	98,9 %
GSM8K	100 / 100	100 %

17 lỗi ban đầu, chỉ 8 do mô hình; 7 do sanitizer của nhóm, sửa trên output thô đã lưu: 98,3 → 98,9 %.

Baseline trên cùng 20 bài

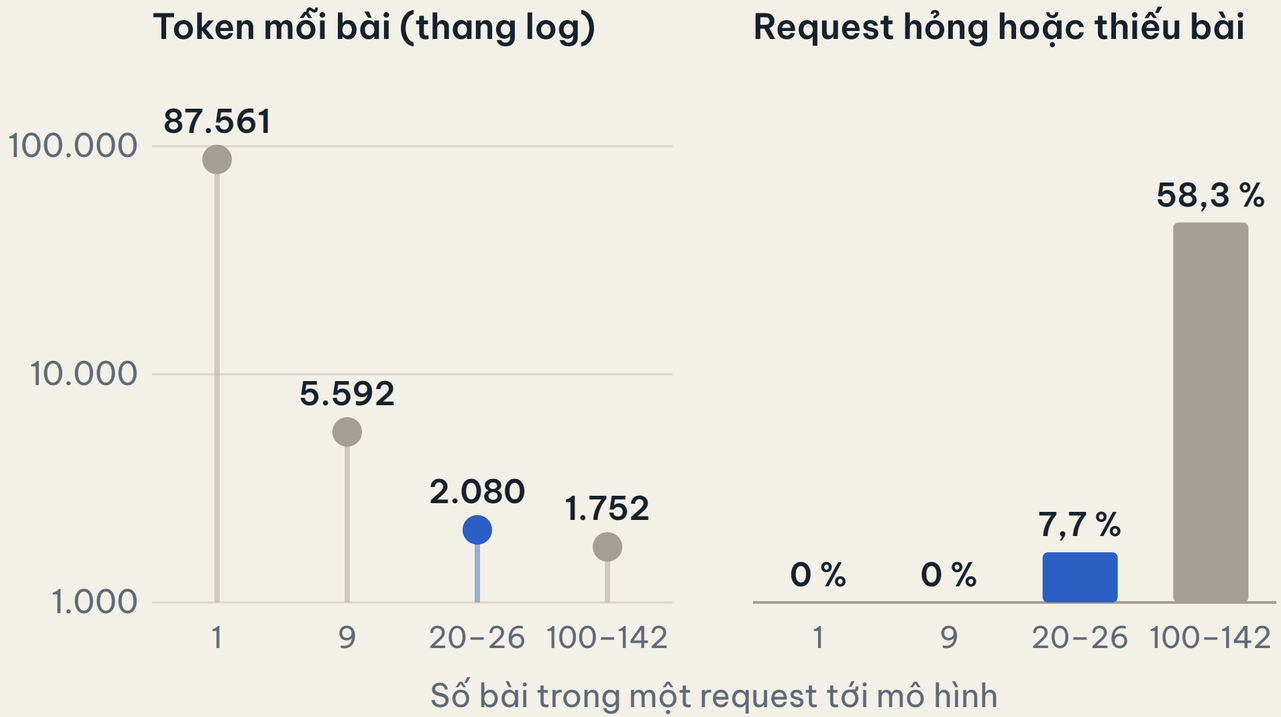
	1 bài mỗi request	Batch job
Compile	20 / 20	20 / 20
Bị gắn cờ ngữ nghĩa	2 / 20	2 / 20
Lean xác nhận đúng	3 / 20	4 / 20
Token mỗi bài	**87.561**	**1.752–2.080**
Thời gian mỗi bài	63 s	5–10 s

Cùng chất lượng, khác chi phí. Compile không còn phân biệt được cấu hình tốt hay xấu: chất lượng thật phải đo bằng ngữ nghĩa.

Số liệu: results/figures/quality_metrics.json · baseline_compare.json · failure_causes_*.json

CÂU HỎI 1 · TẦNG INFERENCE

Gom 20–26 bài mỗi request: rẻ hơn 42 lần, ít hỏng



1 → 4 candidate

Pass@1 98,25 % → Pass@4 100 % với gấp 4 lần token; tỷ lệ bị gắn cờ ngữ nghĩa vẫn 13 %

Few-shot so với zero-shot

Compile 100 % so với 99 %, gắn cờ 13 % so với 17,2 %, gấp 2 lần token

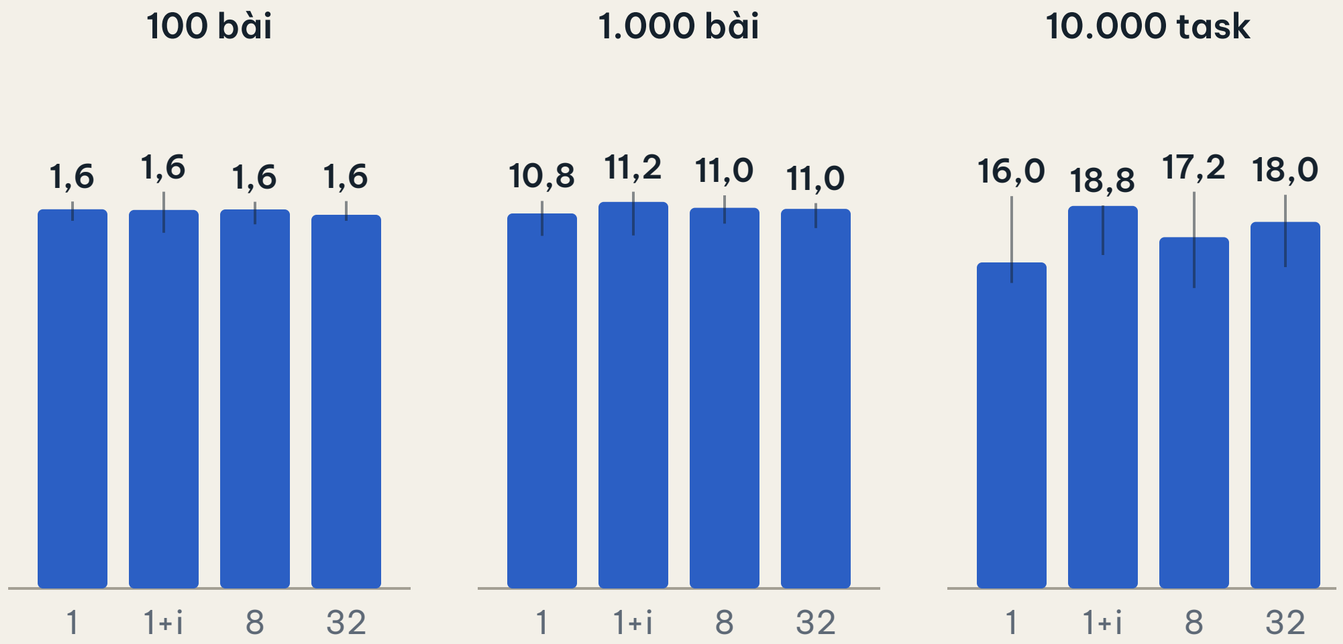
Retry ở tầng inference

Không gửi lại phần thiếu: 56 % ứng viên. Gửi lại theo TaskHash, lô nhỏ hơn: 100 %

Số liệu: results/figures/inference_batching.json · candidates_retry.json · token gồm cả ngữ cảnh cố định (~80K/request)

CÂU HỎI 1 · TẦNG KIỂM CHỨNG

Lô 1, 8 hay 32 task: throughput như nhau ở 2 worker



Số task mỗi lần lease (1+i: lô 1 có index) · bài/s, median 3 lần, vạch mảnh = min–max

Vì sao không khác

Mỗi lần lease chỉ tốn 2,5 ms khi hàng đợi còn 10.000 task. Thời gian nằm ở khởi động REPL (57 s) và 2–6 timeout mỗi lần chạy 10.000 task

Lô 32 ở 100 bài

100 task chỉ thành 4 lô cho 2 worker: cuối lần chạy một worker ngồi không, throughput ổn định 44 so với 58–59 ứng viên/s

Index hàng đợi

Không đổi gì ở 10.000 task. Ở 100.000 task, thiếu index mỗi lease mất 97 ms, có index 0,06 ms

Số liệu: results/raw/batch_runs.csv · batch_median.csv · bench_queue.json · máy đợt 26/10

KHẢ NĂNG MỞ RỘNG · VAI TRÒ BIG DATA

Ở 10.000 task, kết luận khác hẳn ở 100 task

442×

REPL nóng so với chạy lạnh: 45,8 ms so với 20,2 s mỗi ứng viên

→ **worker phải giữ trạng thái**

2,8 → 22,3

bài/s của 1 worker khi tăng 100 → 1K → 10K task

→ **khởi động chỉ khấu hao ở quy mô lớn**

0,1 %

10 ứng viên bệnh lý gây 7–10 lần khởi động lại REPL mỗi lần chạy 10K với 2–4 worker

→ **sự kiện hiếm thành chắc chắn**

26 %

thời gian Lean dồn vào 1 % task chậm nhất; 8 worker: chia tĩnh 14,7 s, work stealing 10,0 s

→ **cần work stealing**

45 %

thời gian worker dành cho khởi động lại REPL ở 10.000 task, 2 worker, timeout 10 s; kiểm chứng thật chỉ 46 %

→ **giả thuyết H1 đúng**

97 → 0,06 ms

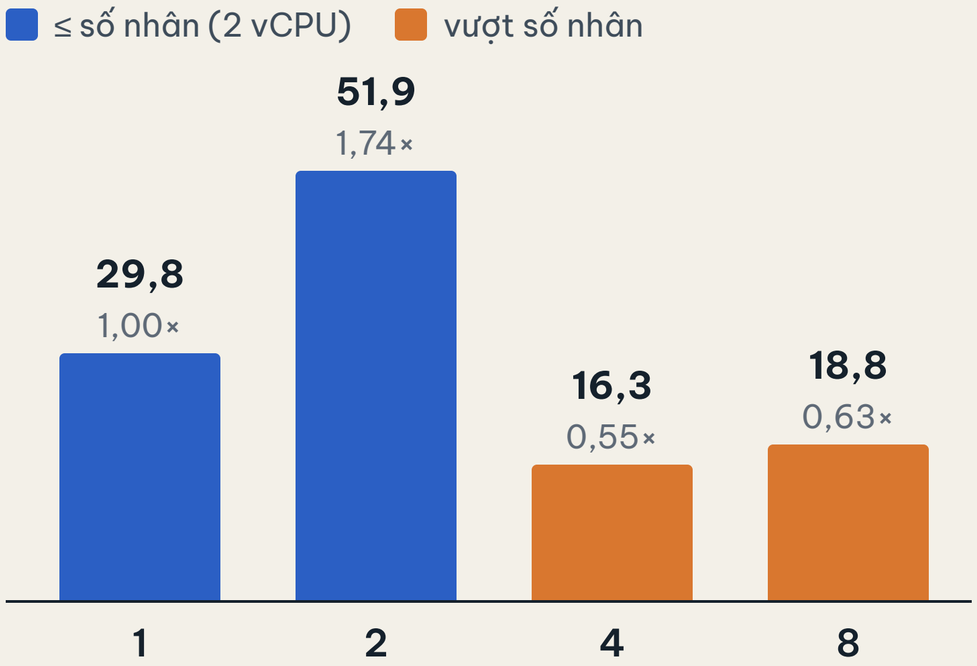
mỗi lần lease ở 100K task chờ: hàng đợi thiếu index so với có index

→ **hàng đợi nghẽn trước cả Lean**

Số liệu: results/figures/matrix_median.csv · time_budget_T10K.json · cold_vs_warm.json · skew_*.json

KHẢ NĂNG MỞ RỘNG · SONG SONG HÓA TRÊN MỘT MÁY

Tăng tốc đến đúng số nhân, sau đó sụp



Throughput ổn định (ứng viên/s) theo số Lean worker · 10.000 task, lô 8, wall timeout 30 s

GIẢ THUYẾT TUẦN 2 → SỐ ĐO

“Nhiều worker thì tràn RAM”

RAM pool ~5 GB với 1–4 worker vì 4,2 GB .olean dùng chung; giới hạn đầu tiên là CPU. Từ 4 worker page cache bắt đầu bị đẩy ra (4,4 → 2,8 GB thường trú); 8 worker: 1,9 GB, swap 3 GB.

“Timeout 10 s giải phóng tài nguyên”

Mỗi timeout kéo theo khởi động lại REPL 30–200 s. Để heartbeat làm giới hạn chính: 24,2 → 51,9 ứng viên/s.

“Gom lô lớn thì tràn VRAM”

Không có GPU; giới hạn thật là 64.000 token đầu ra mỗi lần trả lời: 7/12 request 100–142 bài hỏng. Chia lô theo token.

Số liệu: results/figures/scaling_median.csv · results/raw/runs/sc_T10K_*/memory.csv · máy 2 vCPU, 7,8 GB RAM

CÂU HỎI 2 · LEAN CHẤP NHẬN NHƯNG SAI NGHĨA

Phát hiện sai nghĩa bằng ba lớp kiểm tra

GSM8K · CHỈ CÒN PHÉP TÍNH

```
theorem cand_1ea9da26c3e8 :
  (18 : ℝ) - (5 + 7) = 6 := by sorry
```

Bài bán kẹo chỉ còn một phép trừ: đúng, compile được, nhưng mất hết biến và giả thiết của đề.

MATH · GIẢ THIẾT VÔ NGHIỆM TRÊN ℝ

```
(a b : ℝ) (h1 : a + b = 7)
  (h2 : a ^ 3 + b ^ 3 = 42) :
  1 / a + 1 / b = 21 / 43
```

Không có a, b thực nào thoả giả thiết nên định lý đúng một cách rỗng. Kiểm tra tự động bỏ sót, audit thử bắt được.

1 · Lean tự kiểm tra

Giả thiết mâu thuẫn? Phản chứng được kết luận? Tự suy ra đáp án? Tương đương phát biểu gold?

Cờ do Lean chứng minh là chắc chắn

2 · Heuristic rẻ

Đáp án có nằm trong phát biểu? Bài lời văn còn biến không? Đáp án âm mà phát biểu chỉ dùng ℕ?

Bộ lọc ưu tiên, precision 0,77

3 · Người chấm

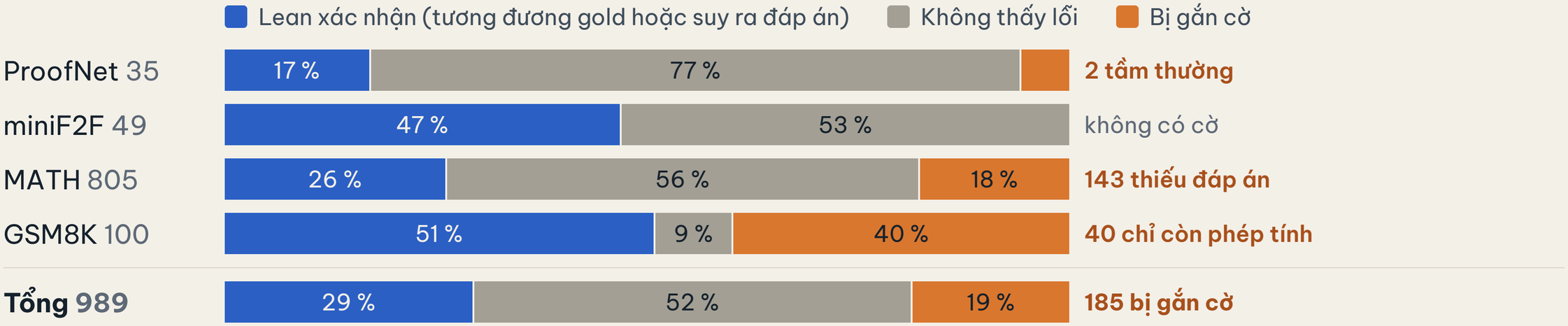
100 mẫu phân tầng theo nguồn, 3 nhãn ĐÚNG / GẦN ĐÚNG / SAI, hai người chấm độc lập

Độ tin cậy đo bằng Cohen's κ

Ví dụ thật: gsm8k:train:04141 · math:test/algebra/2167

CÂU HỎI 2 · KẾT QUẢ

18,7 % phát biểu compile được bị gắn cờ



κ = 0,83

đúng / không giữa 2 LLM chấm thử 100 mẫu; audit hai thành viên: phiếu sẵn

0,73 / 0,58

precision / recall của kiểm tra tự động so với người chấm thử; bỏ sót lỗi cần hiểu toán

100 %

lỗi chèn cố ý bị bắt khi có gold; không có gold, đáp án sai chỉ bị bắt 26 %

Số liệu: results/figures/semantic_summary.json · semantic_layers.json · audit_kappa_*.json · mutation_summary.json

CHỊU LỖI VÀ RETRY

Giết worker giữa chừng: đủ kết quả, không trùng

Giết 1 worker sau 300 kết quả	Giao tối đa 3 lần	Không retry
Bài có kết quả	**1.000 / 1.000**	**995 / 1.000**
Task vào dead-letter	0	8
Dòng kết quả trùng	0	0
Makespan (median 3 lần)	99,7 s	88,0 s

Retry tốn thêm khoảng 13 % thời gian; không retry thì mỗi lần giết worker mất 0,5 % kết quả. Chạy lại không gọi lại mô hình: ứng viên cache theo TaskHash.

Trả lô về hàng đợi khi khởi động lại REPL

Makespan 106 s so với 147 s (1.000 task, timeout 10 s, median 3 lần mỗi bên): worker kia làm nốt lô thay vì chờ REPL mới nạp lại Mathlib

Sự cố thật ở 8 worker

2 REPL không khởi động lại kịp, 2 worker chết; 6 worker còn lại vẫn xong đủ 10.000 kết quả

Nhiều máy qua Redis

SQLite 29,0 và Redis 27,5 ứng viên/s; worker tham gia sau từ tiến trình độc lập làm 376/1.000 task

Số liệu: results/figures/retry_ablation.json · release_ablation.json · fault_tolerance.json · broker_comparison.json

KẾT LUẬN

Trả lời hai câu hỏi của đề bài

Q1 **Batching có lợi ở inference, gần như trung tính ở kiểm chứng**

Gom 20–26 bài mỗi request rẻ hơn 42 lần; lô kiểm chứng 1, 8, 32 cho throughput như nhau; 4 candidate nâng Pass@k lên 100 % nhưng không làm phát biểu đúng nghĩa hơn.

Q2 **Compile được chưa phải đúng nghĩa**

98,9 % compile nhưng 18,7 % bị gắn cờ. Có gold thì bắt mọi lối cài sẵn; so với người chấm thử, kiểm tra tự động chỉ bắt 58 % phát biểu không được chấm ĐÚNG.

BD **Ở 10.000 task, sự kiện hiếm và số nhân là điểm nghẽn**

Mỗi timeout kéo theo khởi động lại REPL; vượt số nhân thì phải thêm máy qua Redis. TaskHash + lease + UPSERT cho kết quả đúng một lần dù worker chết.

BÀN GIAO

Mã nguồn + 26 test · corpus 990 bài · data card, system card, AI-use · 98 run record · demo NL → Lean → checker

GIỚI HẠN VÀ VIỆC TIẾP THEO

Audit 100 mẫu chờ hai thành viên chấm · máy đợt 26/10 chậm hơn 16/10, chỉ so trong cùng đợt · chưa chạy nhiều máy thật

Chi tiết: báo cáo mục 8 · H1, H2 đúng; H3 đúng, nhưng ở tầng kiểm chứng lợi ích quá nhỏ để đổi throughput

PHÂN VAI VÀ SỬ DỤNG AI

Mỗi vai trò có người chính và người dự phòng

Vai trò	Nhóm	Người chính	Dự phòng
Data / evaluation	Data	Nguyễn Mai Hạnh	Bùi Giang Linh
Prompt & model	Model	Lê Quang Đức	Nguyễn Việt Hưng
Batch scheduler	System	Nguyễn Việt Hưng	Lê Quang Đức
Lean worker pool	System	Kim Anh Dũng	Vũ Quốc Hùng
Semantic audit	Model	Bùi Giang Linh	Nguyễn Mai Hạnh
Metrics / reproducibility	Evaluation	Vũ Quốc Hùng	Kim Anh Dũng

Bằng chứng đóng góp: lịch sử Git, issue, pull request và nhật ký trong contributions.md. Mỗi người trả lời được toàn bộ pipeline.

KHAI BÁO AI

Claude: trợ lý lập trình và viết; nhóm giao việc, duyệt từng bước

Claude Haiku 5.5: mô hình sinh mã được đánh giá

Claude Sonnet, Opus: người chấm thử, không thay audit hai người

Mô hình sinh định lý sai thì nhóm chịu trách nhiệm: không phát biểu nào được gọi là đúng nghĩa chỉ vì compile.

Chi tiết: contributions.md · docs/AI_USE_STATEMENT.md · bảng phân vai là đề xuất, nhóm xác nhận trước khi nộp

NHÓM 5 · INT4418

Cảm ơn thầy cô và các bạn đã lắng nghe

Hỏi đáp · demo

> `python -m afp.demo_one --problem gsm8k:train:04141`
>
> NL → Lean → checker cho một bài: ứng viên đã lưu, Lean kiểm trong REPL nóng, kết luận ngữ nghĩa
>
> `python -m afp.demo --n 20 --workers 2 --kill-after 8`
>
> 20 bài, giết 1 worker giữa chừng: task được giao lại, không mất, không trùng

Bàn giao: mã nguồn có test · configs · data sample + manifest · corpus 990 bài · log thô và run record mọi lần chạy · báo cáo · kịch bản demo docs/DEMO_SCRIPT.md